

AI WEATHERWISE

# Complete Project Report

*Deliver intelligent weather forecasts and insights*

Brainstorming · Requirements · Design · Planning

Development · Testing · Documentation · Demonstration

Node.js · Express.js · MongoDB · JWT · Google Gemini AI · OpenWeatherMap

SUBMITTED BY

**DEBORAH A**

sac2427csc5470@ssacollegechennai.com

**CATHERIN JESINTHA J**

sac2427csc5355@ssacollegechennai.com

**COWSER FATHIMA N**

sac2427csc5418@ssacollegechennai.com

**DEEPIKA S**

sac2427csc5467@ssacollegechennai.com

Version 1.0 | 28 September 2026

CONTENTS

# Table of Contents

| Part | Phase                    | What it covers                                               |
|------|--------------------------|--------------------------------------------------------------|
| 1    | Brainstorming & Ideation | Problems, personas, ideas, MoSCoW, solution                  |
| 2    | Requirement Analysis     | Functional and non-functional requirements, data, interfaces |
| 3    | Project Design           | Architecture, MVC, data model, API and security design       |
| 4    | Project Planning         | Epics, sprints, timeline, milestones, risks                  |
| 5    | Project Development      | Setup, implementation, schemas, running the backend          |
| 6    | Project Testing          | Test cases, results, response times, defects                 |
| 7    | Project Documentation    | API reference, setup guide, troubleshooting, future scope    |
| 8    | Project Demonstration    | Demo script, data, checklists, Q&A                           |

## Project at a glance

|            |                                                                                                                               |
|------------|-------------------------------------------------------------------------------------------------------------------------------|
| Project    | AI WeatherWise – a RESTful backend delivering real-time weather with AI summaries and recommendations                         |
| Technology | Node.js, Express.js, MongoDB (Mongoose), JWT, bcryptjs, Google Gemini AI, OpenWeatherMap                                      |
| Scope      | User authentication, favorite locations, weather retrieval, AI insights, security and error handling (API-only first release) |

## PART 1 OF 8

# Brainstorming & Ideation

*Problem discovery, idea generation and solution selection*

## Purpose

Captures how the idea was born, which problems it solves, who it is built for, and which ideas were chosen for the first release.

## Problem statement

Alex, a travel enthusiast, visits many cities and needs to track local weather and receive activity suggestions. Existing tools return raw numbers and leave interpretation to the user.

| #  | Problem                                                    | Impact on the user                                              |
|----|------------------------------------------------------------|-----------------------------------------------------------------|
| P1 | Hard to track and manage many favorite locations           | Users re-search the same cities; no single place to manage them |
| P2 | No personalized clothing or outdoor-activity advice        | Users must decide on their own from numbers                     |
| P3 | Need natural-language summaries, not raw metrics           | Temperature, humidity and wind take effort to interpret         |
| P4 | No secure way to store and sync preferences across devices | Saved cities are lost when the device changes                   |

## Target users

| Persona                            | Goals and needs                                                                                     |
|------------------------------------|-----------------------------------------------------------------------------------------------------|
| Alex – Registered user (traveller) | Track several cities, get clothing and activity advice, access the same list on any device securely |
| Weather application user (public)  | Read current weather quickly without creating an account                                            |
| Developer / Admin                  | Monitor API health, manage registrations, verify weather and AI integrations                        |

**Empathy map (Alex):** Says “What should I wear in Bangalore today?” *Thinks* numbers alone do not tell me what to do. *Does* checks several weather sites and switches devices. *Feels* frustrated by scattered information.

## Brainstormed ideas

| Theme               | Ideas generated                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------|
| Location management | Save, edit, delete favorite cities; prevent duplicates per user; group by country             |
| Weather data        | Real-time OpenWeatherMap data (temperature, humidity, wind, condition); public endpoint       |
| AI intelligence     | Natural-language summary, clothing advice, activity suggestions via Google Gemini             |
| Security            | JWT, bcrypt hashing, role-based guarding, XSS and NoSQL-injection sanitization                |
| Reliability         | Fallback (mock) mode without API keys; centralized error handling                             |
| Administration      | Account management, API health monitoring, database integrity checks                          |
| Future ideas        | Alerts, subscriptions, weather history, AI model management, usage logs, web/mobile front end |

## Idea prioritization (MoSCoW)

| Priority                | Ideas                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Must have</b>        | Registration and login (JWT); favorite locations CRUD; real-time weather; AI recommendation; fallback mode |
| <b>Should have</b>      | AI weather summary; profile update; centralized error handling; input sanitization                         |
| <b>Could have</b>       | Admin monitoring of API health; role matrix for admin actions                                              |
| <b>Won't have (now)</b> | Alerts, subscriptions, weather history, API usage logs, AI model registry, front-end UI                    |

## Proposed solution

A RESTful backend on Node.js and Express.js with MongoDB/Mongoose. It fetches real-time weather and combines it with Google Gemini AI to give intelligent summaries and personalized recommendations. Users store favorite cities securely and reach them from any device.

- **Centralized:** one service for locations, weather and AI insights.
- **Intelligent:** turns raw metrics into plain-language advice.
- **Secure:** JWT authentication and bcrypt hashing.
- **Resilient:** fallback mode works without external keys.

## Technology decisions

| Need                  | Decision             | Reason                                              |
|-----------------------|----------------------|-----------------------------------------------------|
| <b>Runtime</b>        | Node.js + Express.js | Fast, JavaScript everywhere, large ecosystem        |
| <b>Database</b>       | MongoDB + Mongoose   | Flexible documents, simple user–location references |
| <b>Authentication</b> | JWT + bcryptjs       | Stateless; works across devices                     |
| <b>Weather source</b> | OpenWeatherMap       | Current metrics via simple REST API                 |
| <b>AI engine</b>      | Google Gemini        | Natural-language summaries and recommendations      |
| <b>API testing</b>    | Postman              | Request collections and response verification       |

## Success criteria

- A user can register, log in and receive a JWT.
- A user can save, view, update and delete favorite cities without duplicates.
- Current weather is returned within a second; the AI returns a clear recommendation.
- The API keeps working in fallback mode with no external keys.

## PART 2 OF 8

# Requirement Analysis

*Functional, non-functional, data and system requirements*

**In scope:** authentication, favorite locations, real-time weather, AI summaries and recommendations, security and error handling. **Out of scope (first release):** graphical front end, alerts, subscriptions, weather history.

## Stakeholders and roles

| Role                     | Responsibilities                                                                                         |
|--------------------------|----------------------------------------------------------------------------------------------------------|
| Admin                    | Manage user accounts (create, delete, suspend); oversee API health and logs; monitor database integrity  |
| Registered user          | Maintain favorite locations; request weather; use Gemini summaries and activity tips; manage own profile |
| Weather application user | Consumes the public weather endpoints without an account                                                 |
| Developer                | Builds, tests and verifies weather and AI integrations                                                   |

## Functional requirements

| ID    | Requirement                                                                           | Priority |
|-------|---------------------------------------------------------------------------------------|----------|
| FR-01 | Register with name, email and password                                                | Must     |
| FR-02 | Authenticate by email and password; return a signed JWT                               | Must     |
| FR-03 | Store passwords only as bcrypt hashes                                                 | Must     |
| FR-04 | Add a favorite location (city and country)                                            | Must     |
| FR-05 | View, update and delete own favorite locations                                        | Must     |
| FR-06 | Prevent the same city and country being saved twice by one user                       | Must     |
| FR-07 | Return current weather (temperature, humidity, wind speed, condition) for a city      | Must     |
| FR-08 | Generate an AI recommendation (clothing and activities) from weather data             | Must     |
| FR-09 | Generate a natural-language AI weather summary                                        | Should   |
| FR-10 | Run in fallback (mock) mode when weather or AI keys are missing, marking the response | Must     |
| FR-11 | Validate the JWT on protected routes; reject invalid or missing tokens                | Must     |
| FR-12 | Registered user can update profile details                                            | Should   |
| FR-13 | Admin can manage user accounts and monitor API health                                 | Could    |
| FR-14 | Consistent JSON errors through centralized error handling                             | Should   |

## Non-functional requirements

| Category        | Requirement                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| Security        | JWT authentication; bcrypt hashing; role-based route guarding; sanitization against XSS and NoSQL injection; secrets in .env |
| Performance     | Typical responses within one second locally (observed 90–475 ms)                                                             |
| Reliability     | Fallback mode for external services; server starts only after a successful database connection                               |
| Maintainability | MVC and modular structure: routes, controllers, middleware, models, services, config                                         |
| Portability     | Windows 10/11, macOS or Linux with Node.js                                                                                   |
| Usability       | Consistent JSON using a success flag with data or message                                                                    |
| Data integrity  | Mongoose schemas with required fields, trimming, unique email and unique user–city–country index                             |

## User stories

| ID    | As a...     | I want to...                            | So that...                             |
|-------|-------------|-----------------------------------------|----------------------------------------|
| US-01 | Traveller   | register and log in securely            | my saved cities are protected          |
| US-02 | Traveller   | save favorite cities                    | I can track them quickly               |
| US-03 | Traveller   | see current weather of a city           | I know conditions before I go          |
| US-04 | Traveller   | get AI advice on clothes and activities | I do not have to interpret raw numbers |
| US-05 | Public user | read weather without an account         | I can use the service instantly        |
| US-06 | Admin       | monitor users and API health            | the platform stays reliable            |

## System requirements

| Component        | Requirement                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------|
| Operating system | Windows 10/11, macOS or Linux                                                                       |
| Node.js / npm    | Node.js v16+ and npm v8+                                                                            |
| Gemini SDK       | Google Gemini SDK for natural-language insights                                                     |
| MongoDB          | Local or cloud instance                                                                             |
| Tools            | Postman (latest), Visual Studio Code or similar                                                     |
| Hardware         | Intel Core i5 8th Gen / AMD Ryzen 5 or better; 8 GB RAM minimum (16 GB recommended); 1 GB free disk |

## Data requirements

| Entity   | Fields and rules                                                                                                                                                                        |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User     | _id; name (required, trimmed); email (required, unique, lowercase, valid pattern); password (required, min 6 chars, hashed); role (default “reader”); createdAt / updatedAt (automatic) |
| Location | _id; user (ObjectId ref User, required); city and country (required, trimmed); createdAt. Unique index on { user, city, country }                                                       |

## External interfaces and environment

| Interface / Variable    | Description                                                     |
|-------------------------|-----------------------------------------------------------------|
| OpenWeatherMap API      | Current weather metrics for a city                              |
| Google Gemini AI API    | Summaries and personalized recommendations from live conditions |
| MongoDB                 | Persistent User and Location collections via Mongoose           |
| Client (Postman / HTTP) | Sends JSON (GET, POST, PUT, DELETE), receives JSON              |
| PORT                    | Express listening port (default 5000)                           |
| MONGO_URI               | MongoDB connection string                                       |
| JWT_SECRET              | Secret used to sign and verify JWTs                             |
| GEMINI_API_KEY          | Gemini key (optional – fallback mode if absent)                 |

**Assumption:** MongoDB is running before the server starts. **Constraints:** external services need network access and valid keys; first release is API-only. **Acceptance:** every “Must” requirement (FR-01 to FR-08, FR-10, FR-11) is demonstrated through successful Postman tests.

## PART 3 OF 8

# Project Design

Architecture, data model, API and security design

## Design overview

The application follows the **MVC pattern**. Controllers orchestrate weather fetching from OpenWeatherMap and AI insights from Gemini; the Model layer manages User and Location. As a headless backend, the View layer is the API routing layer.

## System architecture

A client such as Postman sends JSON to the Express gateway. Authentication middleware verifies the JWT on private routes, controllers call the weather and AI services, and Mongoose models persist data in MongoDB.

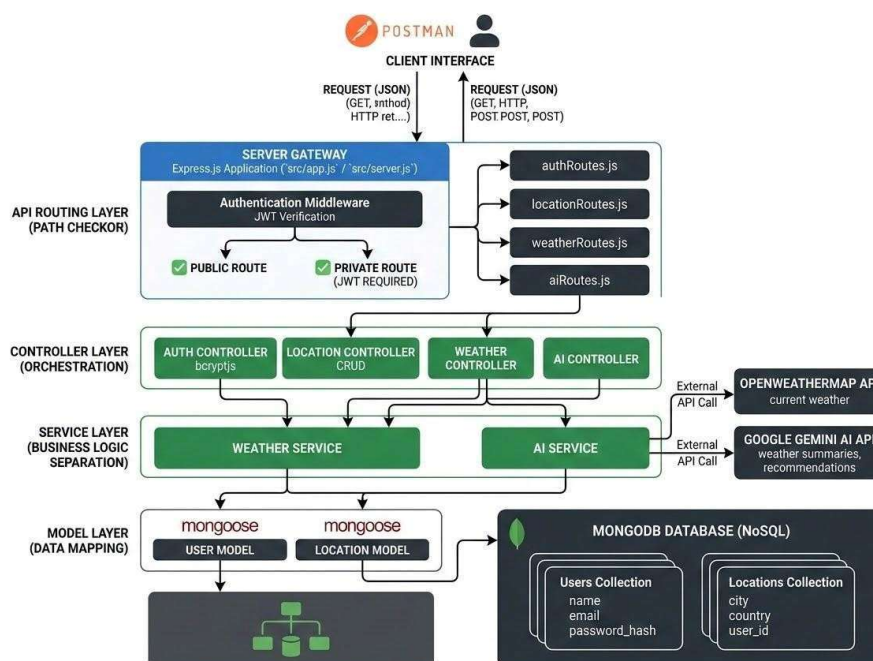


Figure: Layered architecture of AI WeatherWise

| Layer       | Components                                                          | Responsibility                                                 |
|-------------|---------------------------------------------------------------------|----------------------------------------------------------------|
| API routing | authRoutes, locationRoutes, weatherRoutes, aiRoutes; JWT middleware | Maps paths to controllers; separates public and private routes |
| Controller  | Auth, Location (CRUD), Weather, AI controllers                      | Orchestrates requests, validates input, formats JSON           |
| Service     | Weather service, AI service                                         | Business logic and external API calls                          |
| Model       | User and Location (Mongoose)                                        | Schema, validation and data mapping                            |
| Database    | MongoDB – Users and Locations                                       | Persistent storage                                             |

## Key components

- **Express gateway:** listens on the port, maps GET/POST/PUT/DELETE, parses JSON and handles CORS.
- **Auth and role middleware:** extracts and validates the token, checks role for guarded routes.
- **Granular services:** isolated modules for weather retrieval and AI generation.

- **Database interface:** Mongoose translates application logic into MongoDB queries.

## Data design

The extended ER diagram includes entities reserved for future releases (weather data, AI insights, alerts, subscriptions, API usage logs, AI models). The current release implements **User** and **Location** in a one-to-many relationship.

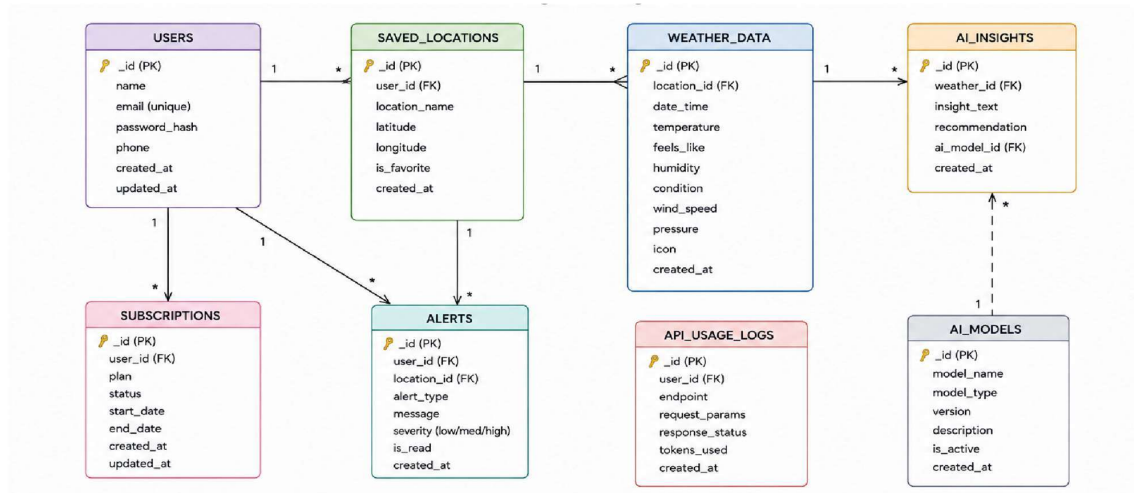


Figure: Entity-Relationship diagram

## API design

| Method       | Endpoint                       | Access | Purpose                                                                 |
|--------------|--------------------------------|--------|-------------------------------------------------------------------------|
| POST         | /api/auth/register             | Public | Register; returns id, name, email, JWT (201)                            |
| POST         | /api/auth/login                | Public | Authenticate; returns user data and JWT (200)                           |
| GET          | /api/weather/:city             | Public | city, temperature, humidity, windSpeed, condition, isMock               |
| POST         | /api/ai/weather-recommendation | JWT    | Takes city, temperature, humidity, condition; returns AI recommendation |
| POST / GET   | /api/locations                 | JWT    | Add / list favorite cities (proposed route)                             |
| PUT / DELETE | /api/locations/:id             | JWT    | Update / remove a favorite city (proposed route)                        |

Every response is JSON with a **success** flag, followed by **data** (or **recommendation**) on success and **message** on failure.

## User flow

1) Register or log in. 2) Save a favorite city. 3) Fetch its current weather. 4) Request AI insights from that data. 5) System validates the JWT and returns results.

## Security and resilience design

- **Authentication:** JWT issued at login/registration, verified by middleware on private routes.
- **Passwords:** bcryptjs hashing; minimum six characters.
- **Authorization:** role-based route guarding.
- **Input safety:** sanitization against XSS and NoSQL injection.
- **Secrets:** JWT secret and API keys in .env, excluded from Git.
- **Fallback mode:** mock data when keys are absent; weather response carries isMock: true.
- **Fail-fast start-up:** server starts only after MongoDB connects; otherwise exits with an error.

- **Centralized error handler:** uniform JSON error responses.

## Project structure

| Folder / file                | Content                                                             |
|------------------------------|---------------------------------------------------------------------|
| <b>src/config</b>            | db.js – MongoDB connection                                          |
| <b>src/controllers</b>       | authController, locationController, weatherController, aiController |
| <b>src/middleware</b>        | authMiddleware – JWT verification                                   |
| <b>src/models</b>            | User.js, Location.js                                                |
| <b>src/routes</b>            | authRoutes, locationRoutes, weatherRoutes, aiRoutes                 |
| <b>src/services</b>          | Weather and AI service modules                                      |
| <b>src/app.js, server.js</b> | Express configuration and server start-up                           |
| <b>Root</b>                  | .env, .gitignore, package.json, postman_collection.json, README.md  |

## PART 4 OF 8

# Project Planning

*Epics, sprints, timeline, resources and risks*

## Deliverables

| #  | Deliverable          | Description                                                   |
|----|----------------------|---------------------------------------------------------------|
| D1 | Backend source code  | Node.js / Express / MongoDB API in a public GitHub repository |
| D2 | Phase-wise documents | Eight documents, from brainstorming to demonstration          |
| D3 | Postman collection   | postman_collection.json with all tested requests              |
| D4 | API testing video    | Recording of the Postman tests                                |
| D5 | Demo video           | Backend demonstration video                                   |

## Epics

| Epic | Title                  | Main tasks                                                  | Output                       |
|------|------------------------|-------------------------------------------------------------|------------------------------|
| 1    | Project Architecture   | Layers, MVC, components, ER model, user flow                | Architecture and ER diagrams |
| 2    | Setup & Configuration  | Folder, npm init, dependencies, .env, .gitignore, structure | Project skeleton             |
| 3    | Backend Development    | Entry point, app, routes, controllers, middleware, services | Working REST API             |
| 4    | Database Configuration | Mongoose connection; User and Location schemas              | Persistent data layer        |
| 5    | Execution & Testing    | Run server; test register, login, weather, AI in Postman    | Test evidence and videos     |

## Sprint plan (6 weeks)

| Sprint     | Focus          | Tasks                                                                                                | Points |
|------------|----------------|------------------------------------------------------------------------------------------------------|--------|
| 1 (Wk 1–2) | Foundation     | Ideation, requirements, architecture and data design; setup, dependencies, .env                      | 13     |
| 2 (Wk 3–4) | Core backend   | MongoDB connection, models, register/login, JWT, location CRUD                                       | 21     |
| 3 (Wk 4–5) | Weather & AI   | Weather service with fallback; Gemini service; recommendation endpoint; error handling, sanitization | 21     |
| 4 (Wk 6)   | Test & deliver | Postman testing, bug fixing, documentation, GitHub upload, demo recording                            | 13     |

## Roles

| Role                      | Responsibility                                            |
|---------------------------|-----------------------------------------------------------|
| Project owner / developer | Overall design, backend implementation, GitHub repository |
| Tester                    | Postman requests, test execution, defects and videos      |
| Documentation lead        | Eight phase documents, README and demo script             |
| Reviewer / mentor         | Reviews each phase deliverable before submission          |

*Tools: Node.js v16+, Express.js, MongoDB with Mongoose, bcryptjs, jsonwebtoken, cors, dotenv, Google Gemini SDK, VS Code, Nodemon, Postman / Thunder Client, Git and public GitHub.*

## Milestones

| Milestone                           | Completion criteria                                   | Target        |
|-------------------------------------|-------------------------------------------------------|---------------|
| <b>M1 – Requirements approved</b>   | Phase 1 and 2 documents reviewed                      | End of Week 2 |
| <b>M2 – Design frozen</b>           | Architecture, ER diagram and API list agreed          | End of Week 3 |
| <b>M3 – Core API working</b>        | Register, login and location CRUD working             | End of Week 4 |
| <b>M4 – AI integration complete</b> | Weather and AI recommendation endpoints working       | End of Week 5 |
| <b>M5 – Delivery</b>                | Tests passed; documents and videos uploaded to GitHub | End of Week 6 |

## Risk management

| Risk                                    | Likelihood | Impact | Mitigation                                      |
|-----------------------------------------|------------|--------|-------------------------------------------------|
| Missing or invalid weather / Gemini key | Medium     | High   | Fallback (mock) mode keeps API functional       |
| MongoDB not running or wrong URI        | Medium     | High   | Fail-fast message; default local URI; checklist |
| Security flaws                          | Low        | High   | Strong JWT secret in .env, sanitization, bcrypt |
| Third-party rate limits or downtime     | Medium     | Medium | Central error handling; fallback data           |
| Schedule slip                           | Medium     | Medium | Prioritize “Must” items; weekly review          |

## Version control and Definition of Done

- Public GitHub repository; submitted phase-wise; .env and node\_modules excluded; small descriptive commits.
- **Done** = code runs with npm start and connects to MongoDB; feature tested in Postman with expected status and body; documented and pushed to GitHub.

PART 5 OF 8

# Project Development

Setup, backend implementation and database configuration

## Project setup

Create the folder *AI Weatherwise*, open it in VS Code, then initialize and install dependencies:

```
npm init -y
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
npm install --save-dev nodemon
```

| Package       | Purpose                          |
|---------------|----------------------------------|
| express       | Web framework and routing        |
| mongoose      | MongoDB object modeling (ODM)    |
| bcryptjs      | Password hashing                 |
| jsonwebtoken  | Create and verify JWTs           |
| cors          | Cross-origin resource sharing    |
| dotenv        | Load variables from .env         |
| multer        | Multipart / file upload handling |
| @google/genai | Google Gemini AI integration     |
| nodemon (dev) | Auto-restart on file changes     |

## Environment (.env)

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

Security tip: never commit .env. Add it to .gitignore so secrets are not published.

## Backend structure

| Module             | Files                                   | Responsibility                                  |
|--------------------|-----------------------------------------|-------------------------------------------------|
| config             | db.js                                   | Creates the MongoDB connection                  |
| controllers        | auth, location, weather, ai controllers | Handle requests and build JSON responses        |
| middleware         | authMiddleware                          | Verifies JWT and protects private routes        |
| models             | User.js, Location.js                    | Mongoose schemas and validation                 |
| routes             | auth, location, weather, ai routes      | Map URL paths to controllers                    |
| services           | Weather and AI services                 | Business logic and external calls with fallback |
| app.js / server.js | –                                       | Express configuration and server start-up       |

## Core implementation

**Entry point:** loads the Express app, connects to MongoDB and listens only after the connection succeeds; on failure it logs the error and exits.

```
connectDB()
  .then(() => app.listen(PORT, () => console.log(`Server running on port ${PORT}`)))
  .catch((error) => {
    console.error('Database connection failed. Express server aborted:', error.message);
    process.exit(1);
  });
```

**Database connection (config/db.js):** uses Mongoose with MONGO\_URI, falling back to the local database `mongodb://127.0.0.1:27017/weatherwise`.

## Schemas

- **Location:** belongs to a user; compound unique index on user + city + country prevents duplicates.
- **User:** validates name, email (unique, lowercase, pattern-checked) and password (min 6); bcryptjs stores hashes only; timestamps enabled.

## Authentication, weather and AI modules

| Module              | How it works                                                                                                       |
|---------------------|--------------------------------------------------------------------------------------------------------------------|
| Registration        | Validates input, hashes password, saves user, returns <code>_id</code> , name, email and JWT (201)                 |
| Login               | Finds user by email, compares password to hash, returns user data and JWT (200)                                    |
| Auth middleware     | Reads token from header, verifies with <code>JWT_SECRET</code> , attaches user to request                          |
| Location controller | Creates, lists, updates and deletes the logged-in user's favorite cities                                           |
| Weather service     | Fetches current weather from OpenWeatherMap; without a key returns mock data with <code>isMock: true</code>        |
| AI service          | Sends temperature, humidity and condition to Gemini; returns a recommendation, or a fallback message without a key |

## Running the backend

Ensure MongoDB is running and `.env` is configured, then run `npm install` and `npm start`. The server connects to MongoDB and listens for requests, ready for Postman or Thunder Client.

## Development notes

- Code default port is 5000; the Postman screenshots use 5001 (PORT=5001 set in `.env` during testing).
- The Gemini package appears as `@google/genai` (install) and `@google/generative-ai` (requirements); keep the name consistent with the service code.
- Passwords must never be logged or returned in responses.

## PART 6 OF 8

# Project Testing

## API test cases, results, evidence and observations

Tests were run in Postman against the local server to verify status codes, response bodies, secure authentication and response times.

| Item              | Detail                                                    |
|-------------------|-----------------------------------------------------------|
| Server / Base URL | Node.js / Express (npm start) – http://localhost:5001/api |
| Database / Tool   | Local MongoDB / Postman (Thunder Client also usable)      |
| Test data         | John Doe, john@mail.com, password 123456, city Bangalore  |
| Mode              | Weather in fallback mode (isMock: true)                   |

**Strategy:** functional API testing, negative testing, security testing (protected routes reject requests without a valid JWT) and performance observation.

### Executed test cases

| ID    | Test case                                               | Expected                 | Actual                                                             | Result |
|-------|---------------------------------------------------------|--------------------------|--------------------------------------------------------------------|--------|
| TC-01 | Register new user – POST /api/auth/register             | 201; user data and token | 201 Created (172 ms); _id, name, email, token                      | Pass   |
| TC-02 | Login – POST /api/auth/login                            | 200; user data and token | 200 OK (90 ms); token returned                                     | Pass   |
| TC-03 | Weather – GET /api/weather/bangalore                    | 200; weather metrics     | 200 OK (305 ms); temp 34, humidity 79, wind 12, Windy, isMock true | Pass   |
| TC-04 | AI recommendation – POST /api/ai/weather-recommendation | 200; recommendation text | 200 OK (475 ms); “Stay hydrated, and wear light cotton clothes.”   | Pass   |

### Planned test cases (pending)

| ID    | Test case                                | Expected result                |
|-------|------------------------------------------|--------------------------------|
| TC-05 | Register with an already used e-mail     | Duplicate error; no new user   |
| TC-06 | Password shorter than 6 characters       | Validation error               |
| TC-07 | Invalid e-mail format                    | Validation error               |
| TC-08 | Login with wrong password                | Authentication error; no token |
| TC-09 | Protected route without token            | 401 Unauthorized               |
| TC-10 | Protected route with invalid token       | 401 Unauthorized               |
| TC-11 | Same city and country twice for one user | Rejected by unique index       |
| TC-12 | NoSQL operators in request body          | Sanitized or rejected          |

## Response times

| API                            | Method | Status      | Time   | Size  |
|--------------------------------|--------|-------------|--------|-------|
| /api/auth/register             | POST   | 201 Created | 172 ms | 556 B |
| /api/auth/login                | POST   | 200 OK      | 90 ms  | 551 B |
| /api/weather/bangalore         | GET    | 200 OK      | 305 ms | 392 B |
| /api/ai/weather-recommendation | POST   | 200 OK      | 475 ms | 348 B |

Sample weather response:

```
{"success":true,"data":{"city":"Bangalore","temperature":34,"humidity":79,"windSpeed":12,"condition":"windy","isMock":true}}
```

## Observations and defects

| #  | Observation                                                   | Suggested action                                                   |
|----|---------------------------------------------------------------|--------------------------------------------------------------------|
| O1 | Weather response is mock data (no live key used)              | Repeat TC-03 with a valid OpenWeatherMap key before the final demo |
| O2 | GET weather request in Postman carries a JSON body            | Remove the body; save the corrected collection                     |
| O3 | Tests used port 5001; code default is 5000                    | Document the PORT value in the README                              |
| O4 | Negative and security tests (TC-05 to TC-12) not yet executed | Run them and add screenshots                                       |

*Test summary: 4 executed, 4 passed, 0 failed, 8 pending. All executed functional tests passed with fast response times. Once the negative and security tests are done, the API can be declared ready for demonstration.*

PART 7 OF 8

# Project Documentation

User, developer and API documentation

## Summary

AI WeatherWise is a RESTful backend on Node.js and Express.js with MongoDB/Mongoose. It delivers real-time weather, and Google Gemini AI adds summaries and personalized recommendations, with JWT authentication, bcrypt hashing, location management, a resilient fallback mode, centralized error handling and request sanitization.

| Problem                                    | Solution provided                                     |
|--------------------------------------------|-------------------------------------------------------|
| Managing many favorite locations           | Location CRUD with per-user storage and no duplicates |
| No personalized outdoor or clothing advice | Gemini recommendations from live weather data         |
| Raw metrics are hard to interpret          | Natural-language summaries and recommendations        |
| No secure cross-device sync                | JWT-secured accounts backed by MongoDB                |

## Key features

- Registration and login with JWT and bcrypt.
- Favorite city CRUD with duplicate prevention.
- Real-time weather: temperature, humidity, wind speed, condition.
- AI summaries and recommendations via Gemini.
- Fallback mode without external keys.
- Role-based route guarding, sanitization and centralized error handling.

## Technology stack

| Layer               | Technology                                   |
|---------------------|----------------------------------------------|
| Runtime / Framework | Node.js v16+, Express.js                     |
| Database            | MongoDB with Mongoose ODM                    |
| Security            | JWT (jsonwebtoken), bcryptjs, cors           |
| AI / Weather        | Google Gemini / OpenWeatherMap               |
| Tools               | Postman, Visual Studio Code, Nodemon, dotenv |

## Installation guide

**Prerequisites:** Node.js v16+, npm v8+, MongoDB (local or cloud), optional Gemini key, Postman.

- 1. Clone or download the repository and open it in VS Code.
- 2. Run `npm install`.
- 3. Create the `.env` file with `PORT`, `MONGO_URI`, `JWT_SECRET`, `GEMINI_API_KEY`.
- 4. Run `npm start`.

## API reference

| Method | Endpoint                       | Auth | Description                         |
|--------|--------------------------------|------|-------------------------------------|
| POST   | /api/auth/register             | No   | Create an account and receive a JWT |
| POST   | /api/auth/login                | No   | Log in and receive a JWT            |
| GET    | /api/weather/:city             | No   | Current weather for a city          |
| POST   | /api/ai/weather-recommendation | JWT  | AI recommendation from weather data |
| CRUD   | /api/locations                 | JWT  | Manage favorite locations           |

Request / response examples:

```
POST /api/auth/register
{ "name": "John Doe", "email": "john@mail.com", "password": "123456" }
-> 201 { "success": true, "data": { "_id": "...", "name": "John Doe",
    "email": "john@mail.com", "token": "<JWT>" } }
```

```
POST /api/ai/weather-recommendation
{ "city": "Bangalore", "temperature": 34, "humidity": 79, "condition": "Windy" }
-> 200 { "success": true,
    "recommendation": "Stay hydrated, and wear light cotton clothes." }
```

## Roles and access

| Role            | Access                                                                            |
|-----------------|-----------------------------------------------------------------------------------|
| Public user     | Public weather endpoints without an account                                       |
| Registered user | Manages favorite locations, requests weather and AI insights, updates own profile |
| Admin           | Manages accounts, monitors API health and database integrity                      |

## Troubleshooting

| Problem                      | Likely cause                           | Solution                                   |
|------------------------------|----------------------------------------|--------------------------------------------|
| "Database connection failed" | MongoDB not running or wrong MONGO_URI | Start MongoDB; check the connection string |
| Port already in use          | Another process uses the port          | Change PORT in .env                        |
| Invalid token                | Missing or expired JWT                 | Log in again; send the token in the header |
| isMock: true in weather      | No live weather API key                | Add a valid key for live data              |
| AI returns a generic message | Missing GEMINI_API_KEY                 | Add the key in .env and restart            |

## Limitations and future enhancements

- No graphical front end yet – API-only.
- Planned entities: weather history, saved AI insights, alerts, subscriptions, API usage logs, AI model registry.
- Weather alerts with severity levels and push or e-mail notifications.
- Admin dashboard for user and API-usage monitoring.
- Automated unit and integration tests; cloud deployment.

*Submission links: [GitHub repository \(public\)](#) and [code drive link](#) to be pasted; videos: [API testing.mp4](#) and [Backend demo.mp4](#).*

## PART 8 OF 8

# Project Demonstration

*Demo plan, script, evidence and submission checklist*

## Purpose

Prepares and records the final demonstration, proving the backend works end to end – from user registration to an AI-generated recommendation.

## Submission requirements

| # | Item              | Requirement                                                                 |
|---|-------------------|-----------------------------------------------------------------------------|
| 1 | GitHub repository | Public link; all project files and documents uploaded; submitted phase-wise |
| 2 | Demo video link   | Recorded demonstration of the working backend                               |

Phase-wise upload: 1 Brainstorming & Ideation, 2 Requirement Analysis, 3 Project Design, 4 Project Planning, 5 Project Development, 6 Project Testing, 7 Project Documentation, 8 Project Demonstration.

## Demonstration prerequisites

- MongoDB running and reachable.
- .env contains PORT, MONGO\_URI, JWT\_SECRET, GEMINI\_API\_KEY.
- Dependencies installed; server starts with npm start and shows the port message.
- Postman collection imported and base URL set; screen recorder and microphone tested.

## Demo flow (6–8 minutes)

| Step | Time | Action                             | What to say                                           |
|------|------|------------------------------------|-------------------------------------------------------|
| 1    | 0:00 | Introduce project and architecture | Problem, solution, MVC layers, technologies           |
| 2    | 1:00 | Show structure in VS Code          | Routes, controllers, middleware, models, services     |
| 3    | 2:00 | Run npm start                      | Connects to MongoDB, then listens on the port         |
| 4    | 3:00 | Register API                       | New user created; 201 Created with a JWT              |
| 5    | 4:00 | Login API                          | Credentials verified; 200 OK with a token             |
| 6    | 5:00 | Weather API                        | Current metrics for Bangalore; note the fallback flag |
| 7    | 6:00 | AI recommendation API              | Gemini advice from the weather data                   |
| 8    | 7:00 | Summary and future scope           | Alerts, subscriptions, front end                      |

## Demo data

| Field                    | Value                                        |
|--------------------------|----------------------------------------------|
| Name / E-mail / Password | John Doe / john@mail.com / 123456            |
| City                     | Bangalore                                    |
| Weather sample           | Temperature 34, humidity 79, condition Windy |

## Video recording checklist

- Close unrelated windows and notifications; use a readable font size.

- Speak clearly and follow the script order; show both request and full response in Postman.
- Keep the video under 10 minutes, export as MP4, upload and share a link anyone can view.

## Expected questions and answers

| Question                                | Answer                                                                    |
|-----------------------------------------|---------------------------------------------------------------------------|
| Why MongoDB?                            | Flexible documents and simple user-to-location references using Mongoose  |
| How is security handled?                | JWT, bcrypt-hashed passwords, role-based guarding, input sanitization     |
| What if a weather or AI key is missing? | Fallback mode returns mock data (isMock: true) so the API keeps working   |
| How does AI add value?                  | Gemini turns raw metrics into clear clothing and activity recommendations |
| What is next?                           | Front end, alerts, subscriptions, weather history, admin dashboard        |

## Backup plan

- Database fails to connect: show the recorded demo and explain fail-fast start-up.
- Network unavailable: fallback mode still returns weather and AI responses.
- Keep screenshots of every API response ready.

## Final submission checklist

- All eight phase documents uploaded to the public GitHub repository.
- Source code, postman\_collection.json and README pushed; .env excluded.
- Repository link is public and opens without login; demo video uploaded and link tested.
- Links pasted into the submission form.

---

## Conclusion

AI WeatherWise demonstrates a secure, AI-enabled weather backend that turns raw weather data into actionable advice. The functional tests for registration, login, weather and AI recommendation passed with response times between 90 and 475 ms. Remaining next steps are the pending negative and security tests, a run with a live weather key, and the future enhancements (alerts, subscriptions, weather history, front end).